

MEDIATEK

MT8676 Android DSI Panel User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Henry Tu	Official release

Table of Contents

Version History	2
Table of Contents	3
1 Overview	4
2 Preconditions for Bring Up	5
2.1 Obtain the Following Information from the Panel Vendor	5
2.2 HW Schematic Review	5
3 Early-Stage Preparation	6
3.1 SW env: LK2	6
3.2 SW env: Kernel-6.1	7
4 Important Notes	9
5 LK Drivers	10
5.1 Add New LCM Driver in LK2	10
5.2 Porting LCM Driver in LK	11
5.3 DSI Command Usage in LK	19
6 Kernel DRM Drivers	21
6.1 Add Kernel LCM Driver in Kernel	21
6.2 Porting LCM Driver in Kernel	24
7 Extremely precise FPS	29
8 Bring up Debug SOP	30
8.1 Panel Not Work in LK Stage	30
8.2 Panel Does Work in the Kernel Stage	31
8.3 Panel Does not Work in Resume Stage	31
8.4 Panel Flicker	32
9 Dump Reg	34
9.1 LK Stage	34
9.2 Kernel Dump Reg	34
9.3 Adb cmd Debug Reg Setting	34
9.4 Test Pattern CMD	35
9.5 Analyze Reg	35
10 How to Use MTK max96789 Panel Driver	38
Exhibit 1 Terms and Conditions	44

1 Overview

This document aims to present a SOP for panel driver porting, along with the associated debugging skills.

2 Preconditions for Bring Up

2.1 Obtain the Following Information from the Panel Vendor

- The panel vendor needs to illuminate the panel on the fixture using an external input signal
- Gets “power on/off sequence, port number, ddic number, cmd/vdo mode, Cphy/dphy, Lane number” and so on
- Initiates/deinitiates cmd (must be the command mentioned in item 1)
- Width/height/vfp/vbp/vsa/hfp/hbp/hsa/fps/mipi clock and so on
- Bist mode cmd (optional)
- Sends cmd to ddic with dcs/generic or another type
- cmd<0xB0 used with DCS; cmd >= 0xB0 used with Generic by default
- DSC parameter, slice number (make sure the SoC supports those DSC parameters first)
- How to implement other functions (backlight/fps change/cabc...)
- If the panel connects to SOC with a bridge, get those settings from the panel vendor or bridge vendor

2.2 HW Schematic Review

- Configures panel power control
 - If HW design uses MTK’s regulator, you can obtain the control function from the PMIC owner (function in LK/Kernel)
 - If HW design uses a 3rd party PMIC, obtain the control function from the 3rd party vendor
- Configures GPIO
 - Configures GPIO in DWS using the GPIO table
- Configures backlight
 - Pulls the backlight control GPIO to high when bringing it up, then configures it to the correct function later

3 Early-Stage Preparation

3.1 SW env: LK2

Note:

- If there is no link, skip this step.

- Adds a new LCM driver

```
project/auto8676p1_64_bsp_6p.mk
---
23 MODULE_DEPS += app/$(BOOT_APP)
24
25 MTK_IMPU_ENABLE := yes
26 MTK_LCM_COMMON_DRIVER_SUPPORT := no
27 MTK_LCM_LIST_SUPPORT="max96789_dsi_vdo"
28 MTK_FORCE_DISABLE_PRIM_DUAL_PIPE = yes
```

- Modifies LCM support list in \$(project).mk

☐	dev/lcm/include/lcm_drv.h
☒	dev/lcm/max96789_dsi_vdo/max96789_dsi_vdo.c
☒	dev/lcm/max96789_dsi_vdo/rules.mk
☐	dev/lcm/mt65xx_lcm_list.c
☐	platform/mediatek/mt6897/disp/disp_lcm.c
☐	project/auto8676p1_64_bsp_6p.mk

- Adds the panel driver ko to ko_order_table in the device folder

```
device/mediateksample/auto8676p1_64_bsp+/8628381/3/ko_order_table.csv
85 mtk_panel_ext.ko,./../kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_panel_ext.ko
,ramdisk,Y,Y,user/userdebug/eng
86 bridge-serdes-max96789.ko,./../kernel_device_modules-6.1/drivers/gpu/drm/panel/bridge-serdes-max96789
.ko,ramdisk,Y,Y,user/userdebug/eng
87 panel-serdes-max96789.ko,./../kernel_device_modules-6.1/drivers/gpu/drm/panel/panel-serdes-max96789.k
o,ramdisk,Y,Y,user/userdebug/eng
```

- Adds the LCM driver setting to the list

```
/quark/lk2+/8627975/13/dev/lcm/mt65xx_lcm_list.c
34
35 LCM_DRIVER *lcm_driver_list[] = {
36
37 #if defined(MAX96789_DSI_VDO)
38     &max96789_dsi_vdo_lcm_drv,
39 #endif
40 }
```

- Adds LCM driver info in lcm_drv.h

```
/quark/lk2+/8627975/13/dev/lcm/include/lcm_drv.h
1082 extern LCM_DRIVER nt36072c_mip_dsi_vdo_120hz_jol
1083 extern LCM_DRIVER nt36532_wqxa2944_dsi_vdo_144hz
1084 extern LCM_DRIVER jd9365da_wxga_dsi_vdo_lcm_drv;
1085 extern LCM_DRIVER max96789_dsi_vdo_lcm_drv;
1086
```

3.2 SW env: Kernel-6.1

Note:

- The settings for Kernel-6.6 are the same as below, but the paths are different

1. Adds a new panel driver

```
gerrit.mediatek.inc/c/quark/kernel_device_modules-6.1/
A drivers/gpu/drm/panel/bridge-serdes-max96789.c
M drivers/gpu/drm/panel/Kconfig
M drivers/gpu/drm/panel/Makefile
A drivers/gpu/drm/panel/panel-serdes-max96789.c
```

2. Adds LCM config in kconfig

```
/quark/kernel_device_modules-6.1/+8705673/8/drivers/gpu/drm/panel/Kconfig
24
25 config DRM_PANEL_SERDES
26 » tristate "serdes max96789 vdo mode panel"
27 » depends on OF
28 » depends on DRM_MIPI_DSI
29 » depends on BACKLIGHT_CLASS_DEVICE
30 » help
31 » Say Y here if you want to enable support for max96789 Serdes
32 » vdo mode panel. The panel has a 1920x1080
33 » resolution and uses 24-bits RGB per pixel. It provides a
34 » MIPI DSI interface to host.
```

3. Adds LCM config in makefile

```
/kernel_device_modules-6.1/+8705673/8/drivers/gpu/drm/panel/Makefile
i9 obj-$(CONFIG_DRM_PANEL_ILI7838E_DV2_SPR_CMD) += k6989v1_64_alpha/panel-ili7838e-dv2
i0 endif
i1
i2 obj-$(CONFIG_DRM_PANEL_SERDES) += panel-serdes-max96789.o bridge-serdes-max96789.o
```

4. Adds LCM config in kernel config

```
kernel_device_modules-6.1/+8705663/2/arch/arm64/configs/mgk_64_k61_defconfig
3 CONFIG_DRM_PANEL_BOE_JD9365DA=m
4 CONFIG_DRM_PANEL_SERDES=m
5 CONFIG_DRM_PANEL_TRULY_TD4330_VD0=m
```

5. Adds LCM config in kleaf

```
/kernel_device_modules-6.1/+8705631/2/kernel/kleaf/mgk_64_k61.bzl
120 "drivers/gpu/drm/mediatek/mediatek_v4/mtk_eync.ko",
127 "drivers/gpu/drm/mediatek/mml/mtk-mml.ko",
128 "drivers/gpu/drm/panel/bridge-serdes-max96789.ko",
129 "drivers/gpu/drm/panel/k6985v1_64_alpha/panel-nt37705-a1
58 "drivers/gpu/drm/panel/panel-samsung-s68fc01-vdo-aod.ko",
59 "drivers/gpu/drm/panel/panel-serdes-max96789.ko",
50 "drivers/gpu/drm/panel/panel-sc-nt36672c-vdo-120hz.ko",
```

6. Adds LCM config in dts

```

12/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_6p.dts
    status = "okay";
};
&i2c1 {
    status = "okay";
    max96789: max96789@48 {
        compatible = "maxim,max96789";
        reg = <0x48>;
        reset-gpios = <&pio 37 0>;
        pinctrl-names = "default";
        setting = <&setting_compatible>;
        init-in-lk = <1>;
    };
};
7 &dsi0 {
    status = "okay";
    #address-cells = <1>;
    #size-cells = <0>;
    panel1@0 {
        compatible = "lcm,dsi,max96789";
        reg = <0>;
        ser = <&max96789>;
    };
    port {
        panel_in1: endpoint {
            remote-endpoint = <&dsi_out>;
        };
    };
};

```

4 Important Notes

- To prevent panel flickering when boot from LK to kernel, settings of DSI/MIPI_TX/panel are all based on LK parameters after kernel startup. Only after suspend/resume, all DSI and panel parameters are obtained from the Kernel. If the parameters after suspend/resume are different from those at power-on startup, please check the differences in these three parts of the settings.
- It is normal for the system to not work properly when the kernel panel driver is not ready, as the Kernel will fetch some parameters for display at startup.
- The parameters marked as 'must set' on the following pages are mandatory, while other parts should be filled in as needed.

5 LK Drivers

- How to determine whether the project uses LK or LK2?
 - The MTK_LK_VERSION in the projectconfig.mk in the Device folder corresponds to the project
- LK2 key path: /vendor/mediatek/proprietary/bootable/bootloader/lk2/
- LK key path: /vendor/mediatek/proprietary/bootable/bootloader/lk/

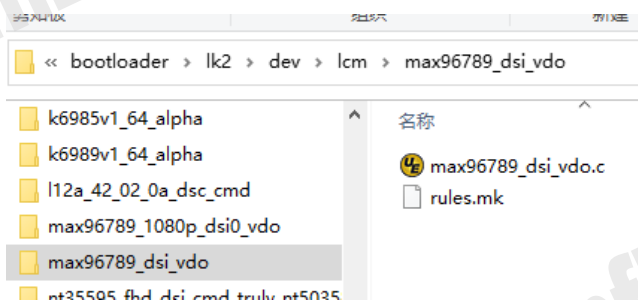
Name	Path
Display Core	platform /mediatek/ mt6xxx /disp
Panel driver	dev / lcm /
LOGO picture	LK2: Vendor/mediatek/proprietary/external/BootLogo/logo LK: vendor/mediatek/proprietary/bootable/bootloader/lk/dev/logo

- MTK key path: /vendor/mediatek/proprietary/bootable/bootloader/lk2/
- OR: /vendor/mediatek/proprietary/bootable/bootloader/lk/

Name	Files
MTK Display code	ddp_xxx, disp_xxx, primary_display.c, mt_disp_drv.c

5.1 Add New LCM Driver in LK2

- Step 1: Adds your <lcm driver>
 - Adds your <lcm driver> into the following path:
 - Creates a new folder under the /vendor/mediatek/proprietary/bootable/bootloader/lk2/dev/lcm/ directory, note that the folder name should be the same as the name added in the second step of the mk file, it is recommended to copy the existing driver and rename it
 - The name of driver (xxx.c) need same as folder name
 - Please refer to [Section 6.2 Porting LCM Driver in Kernel](#) for detail information
 - Takes <max96789_dsi_vdo> for example:



- Step 2: Adds your <lcm config> in <project> makefile
 - Adds your <lcm config> in <project>.mk
vendor\mediatek\proprietary\bootable\bootloader\lk2\project\<project>.mk
 - Takes < max96789_dsi_vdo > for example:

- Single panel setting:

```

27
28 MTK_FORCE_DISABLE_PRIM_DUAL_PIPE = yes
29 MTK_LCM_COMMON_DRIVER_SUPPORT := no
30 MTK_LCM_LIST_SUPPORT="max96789_dsi_vdo"
31 MTK_RT4831A_SUPPORT = 1
32 MTK_SECURITY_SW_SUPPORT := no
33 MTK_SECURITY_ANTI_ROLLBACK := no

```

- Multi-panel setting:

```

27 MODULE_PATH := $(platform)/$(PRODUCT)/common/pvintw
28
29 MTK_FORCE_DISABLE_PRIM_DUAL_PIPE = yes
30 MTK_LCM_COMMON_DRIVER_SUPPORT := no
31 MTK_LCM_LIST_SUPPORT="max96789_dsi_vdo_jd9365da_wxga_dsi_vdo"
32 MTK_RT4831A_SUPPORT = 1
33 MTK_SECURITY_SW_SUPPORT := no
34 MTK_SECURITY_ANTI_ROLLBACK := no

```

- Step 3: Adds your <lcm main structure> into lcm list

- Adds your <lcm main structure> into lcm list in [vendor\mediatek\proprietary\](#):
 - bootable\bootloader\lk2\dev\lcm\mt65xx_lcm_list.c
 - bootable\bootloader\lk2\dev\lcm\include\lcm_drv.h
- Takes < max96789_dsi_vdo > for example:

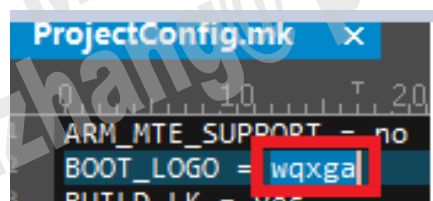
```

1081 extern LCM_DRIVER nt36672e_fhdp_dsi_vdo_120hz_jdi_dphy_e)
1082 extern LCM_DRIVER nt36672e_fhdp_dsi_vdo_120hz_jdi_cphy_e)
1083 extern LCM_DRIVER nt36532_wxga2944_dsi_vdo_144hz_boe_lcr
1084 extern LCM_DRIVER jd9365da_wxga_dsi_vdo_lcm_drv;
1085 extern LCM_DRIVER max96789_dsi_vdo_lcm_drv;
1086
1087
34
35 LCM_DRIVER *lcm_driver_list[] = {
36
37 #if defined(MAX96789_DSI_VDO)
38 &max96789_dsi_vdo_lcm_drv,
39 #endif
40
41 #if defined(MD9365DA_WXGA_DSI_VDO)

```

- Step 4: Switches the logo if the LCM resolution is different

- Modify define marco of BOOT_LOGO in device\mediatekprojects\<project>\projectconfig.mk
- Take < max96789_dsi_vdo > for example:



```

ProjectConfig.mk
...
ARM_MTE_SUPPORT = no
BOOT_LOGO = wxga
BUILD_LK = YES

```

- Step 5: Rebuilds LK

- Rebuilds lk and re-download lk.img.

5.2 Porting LCM Driver in LK

- Step 1: Implements the lcm_drv structure (must set)

- Takes <tv101wum_n16_wuxga_dsi_video_boe_rtq6752.c> for example:

- Step 2: Fills the LCM parameters (must set)

```
LCM_DRIVER tv101wum_n16_wuxga_dsi_vdo_boe_rtq6752_drv = {
    .name = "tv101wum_n16_wuxga_dsi_vdo_boe_rtq6752_drv",
    .set_util_funcs = lcm_set_util_funcs,
    .get_params = lcm_get_params,
    .init = lcm_init,
    .suspend = lcm_suspend,
    .resume = lcm_resume,
    .init_power = lcm_init_power,
    .compare_id = lcm_compare_id,
    .resume_power = lcm_resume_power,
    .suspend_power = lcm_suspend_power,
    .ata_check = lcm_ata_check,
    .update = lcm_update,
};
```

- Configures the basic information according to the HW connection, LCM type, DSI mode, LCM size, and PLL in the lcm_get_params function

- LCM width/height

```
static void lcm_get_params(LCM_PARAMS *params)
{
    memset(params, 0, sizeof(LCM_PARAMS));

    params->type = LCM_TYPE_DSI;

    params->width = FRAME_WIDTH;
    params->height = FRAME_HEIGHT;
```

- LCMparams (dsi setting, must set)

```
params->dsi.mode = SYNC_PULSE_VDO_MODE;
params->dsi.switch_mode = CMD_MODE;
params->dsi.switch_mode_enable = 0;

/* DSI */
/* Command mode setting */
params->dsi.LANE_NUM = LCM_FOUR_LANE;
/* The following defined the format for data coming from LCD engine.
params->dsi.data_format.color_order = LCM_COLOR_ORDER_RGB;
params->dsi.data_format.trans_seq = LCM_DSI_TRANS_SEQ_MSB_FIRST;
params->dsi.data_format.padding = LCM_DSI_PADDING_ON_LSB;
params->dsi.data_format.format = LCM_DSI_FORMAT_RGB888;

/* Highly depends on LCD driver capability. */
params->dsi.packet_size = 256;
```

- LCMparams (video timing, must set)

```
params->dsi.vertical_sync_active = VSA;
params->dsi.vertical_backporch = VBP;
params->dsi.vertical_frontporch = VFP;
params->dsi.vertical_frontporch_for_low_power = 750;
params->dsi.vertical_active_line = FRAME_HEIGHT;

params->dsi.horizontal_sync_active = HSA;
params->dsi.horizontal_backporch = HBP;
params->dsi.horizontal_frontporch = HFP;
params->dsi.horizontal_active_pixel = FRAME_WIDTH;
params->dsi.ssc_disable = 1;
```

- esd config (optional)

```
params->dsi.esd_check_enable = 1;
params->dsi.customization_esd_check_enable = 0;
params->dsi.lcm_esd_check_table[0].cmd = 0x0A;
params->dsi.lcm_esd_check_table[0].count = 1;
params->dsi.lcm_esd_check_table[0].para_list[0] = 0x9C;
```

- PLL configuration (PLL_CLOCK and data_rate must set at least one)

```
00179: #if.(LCM_DSI_CMD_MODE)
00180: ....params->dsi.PLL_CLOCK.=.551;.
00181: #else
00182: ....params->dsi.PLL_CLOCK.=.270;.
00183: #endif
00184: ....params->dsi.data_rate.=.563;
```

(DPHY)
$$\text{Data Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Blanking Ratio} \times \text{Bit Per Pixel}}{\text{Lane Num}}$$

(DPHY DSC)
$$\text{Data Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Compression Ratio} \times \text{Bit Per Pixel} + \text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times (\text{Blanking Ratio} - 1) \times \text{Bit Per Pixel}}{\text{Lane Num}}$$

(CPHY)
$$\text{Symbol Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Blanking Ratio} \times \text{Bit Per Pixel} \times 7/16}{\text{Trio Num}}$$

(CPHY DSC)
$$\text{Symbol Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Compression Ratio} \times \text{Bit Per Pixel} \times 7/16 + \text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times (\text{Blanking Ratio} - 1) \times \text{Bit Per Pixel} \times 7/16}{\text{Trio Num}}$$

- ssc config (optional)

MIPI Clock Spread Spectrum:

```
params->dsi.ssc_disable = 0;
params->dsi.ssc_range = 4;
```

Ssc_disable: 0 → enable ssc, 1 → disable ssc

Ssc_range: 1~8, default value:0

Mipi clk mode setting:

- a. non-continuous clk mode:

```
params->dsi.cont_clock = 0;
params->dsi.clk_lp_per_line_enable = 1;
```

- b. continuous clk mode:

```
params->dsi.cont_clock = 1;
/* params->dsi.clk_lp_per_line_enable = 1; */
```

- MiPi clock mode config (optional)
- cphy/dphy config (by case)
- Dual port setting (by case)
- Sets the cmd or vdo mode depending on the panel. Please ask the panel vendor how to set it

```
params->lcm_if = LCM_INTERFACE_DSI0;
params->lcm_cmd_if = LCM_INTERFACE_DSI_DUAL;

params->dsi.mode = CMD_MODE;
params->dsi.output_mode = MTK_PANEL_DUAL_PORT;
params->dsi.ssc_disable = 1;
```

- DSC parameter configuration: obtain parameters from the vendor.

```
params->dsi.IsCphy = 1;
```

- If DSC needs to be enabled, DSC must be enabled and DSC parameters must be set.
- DSC enablement depends on the panel, and DSC parameters need to be provided by the panel vendor.
- MTK specific parameter dsc_cfg: Set to 0x22 for 8bpc to 8bpp, set to 0x828 for 10bpc to 8bpp.
- Other parameters correspond one-to-one with the parameters provided by the panel vendor.
- Note that the range of rc_buf_thresh is 14-126. If the vendor's data is much larger than this, shift the data to the right with 6 bits

- Demo for DCS settings

```

params->dsi.dsc_enable = 1;
params->dsi.dsc_params.ver = 17;
params->dsi.dsc_params.slice_mode = 1;
params->dsi.dsc_params.rgb_swap = 0;
params->dsi.dsc_params.dsc_cfg = 34;
params->dsi.dsc_params.rct_on = 1;
params->dsi.dsc_params.bit_per_channel = 8;
params->dsi.dsc_params.dsc_line_buf_depth = 9;
params->dsi.dsc_params.bp_enable = 1;
params->dsi.dsc_params.bit_per_pixel = 128;
params->dsi.dsc_params.pic_height = 2400;
params->dsi.dsc_params.pic_width = 1080;
params->dsi.dsc_params.slice_height = 8;
params->dsi.dsc_params.slice_width = 540;
params->dsi.dsc_params.chunk_size = 540;
params->dsi.dsc_params.xmit_delay = 170;
params->dsi.dsc_params.dec_delay = 526;
params->dsi.dsc_params.scale_value = 32;
params->dsi.dsc_params.increment_interval = 43;
params->dsi.dsc_params.decrement_interval = 7;
params->dsi.dsc_params.line_bpg_offset = 12;
params->dsi.dsc_params.nfl_bpg_offset = 3511;
params->dsi.dsc_params.slice_bpg_offset = 3255;
params->dsi.dsc_params.initial_offset = 6144;
params->dsi.dsc_params.final_offset = 7072;
params->dsi.dsc_params.flatness_minqp = 3;
params->dsi.dsc_params.flatness_maxqp = 12;
params->dsi.dsc_params.rc_model_size = 8192;
params->dsi.dsc_params.rc_edge_factor = 6;
params->dsi.dsc_params.rc_quant_incr_limit0 = 11;
params->dsi.dsc_params.rc_quant_incr_limit1 = 11;
params->dsi.dsc_params.rc_tgt_offset_hi = 3;
params->dsi.dsc_params.rc_tgt_offset_lo = 3;

```

- Vendor dsc setting example 1

800 2560 slice 20	Dec	Hex
force reg_dsc_version_major[3:0] =	1	1
force reg_dsc_version_minor[3:0] =	1	1
force reg_pps_identifier[7:0] =	0	00
force reg_bits_per_component[3:0] =	8	8
force reg_linebuf_depth[3:0] =	9	9
force reg_block_pred_enable =	1	1
force reg_convert_rgb =	1	1
force reg_simple_422 =	0	0
force reg_vbr_enable =	0	0
force reg_bits_per_pixel[9:0] =	128	80
force reg_pic_height[15:0] =	2560	0A00
force reg_pic_width[15:0] =	800	0320
force reg_slice_height[15:0] =	20	14
force reg_slice_width[15:0] =	800	0320
force reg_chunk_size[15:0] =	800	0320
force reg_initial_xmit_delay[9:0] =	512	0200
force reg_initial_dec_delay[15:0] =	657	0291
force reg_initial_scale_value[5:0] =	32	0020

- Vendor dsc setting example 2

```

unsigned char pps[] = {0x11, 0x00, 0x00, 0xAB, 0x30, 0x80, 0x07, 0x54,
0x0B, 0xB8, 0x00, 0x14, 0x02, 0xEE, 0x02, 0xEE,
0x02, 0x00, 0x01, 0xBC, 0x00, 0x20, 0x00, 0x90,
0x00, 0x0A, 0x00, 0x06, 0x10, 0x00, 0x12, 0x4E,
0x18, 0x00, 0x10, 0xF0, 0x07, 0x10, 0x20, 0x00,
0x06, 0x0F, 0x0F, 0x33, 0x0E, 0x1C, 0x2A, 0x38,
0x46, 0x54, 0x62, 0x69, 0x70, 0x77, 0x79, 0x7B,
0x7D, 0x7E, 0x02, 0x02, 0x22, 0x00, 0x2A, 0x40,
0x2A, 0xBE, 0x3A, 0xFC, 0x3A, 0xFA, 0x3A, 0xF8,
0x3B, 0x38, 0x3B, 0x78, 0x3B, 0xB6, 0x4B, 0xB6,
0x4B, 0xF4, 0x4B, 0xF4, 0x6C, 0x34, 0x84, 0x74,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};

```

- GPIO configuration (refer to GPIO SOP)

	ADC	ClockBuffer	EINT	GPIO	I2C	KEYPAD	MD1_EINT	PMIC		
	ID	EintMode	Def.Mode	InPull En	InPull SelHigh	Def.Dir	OutHigh	VarName1	VarName2	
39	GPIO39	☐	4:MD_UTXD1	☐	☐					
40	GPIO40	☐	1:DISP_PWM	☐	☐			GPIO_PWM_BL_PIN		
41	GPIO41	☐	1:DSI_TE	☒	☐			GPIO_DSI_TE_PIN		
42	GPIO42	☐	0:GPIO42			OUT	☐	GPIO_LCM_RST		
43	GPIO43	☐	1:MD_INT1_...	☒	☒			GPIO_SIM1_HOT_PLUG		
44	GPIO44	☐	NC	☒	☐	IN	☐			
45	GPIO45	☐	1:MD1_SIM...	☐	☐			GPIO_SIM2_SCLK		
46	GPIO46	☐	1:MD1_SIM...	☐	☐			GPIO_SIM2_SRST		

```

static void lcm_set_reset_pin(UINT32 value)
{
    int rst_pin;

    rst_pin = lcm_util_get_pin("lcm_rst_out0_gpio");
    if (rst_pin < 0) {
        OUTREG32(MMSYS_CONFIG_BASE+0x150, value);
        dprintf(0, "reg set lcm rst pin\n");
    } else {
        dprintf(0, "gpio set lcm rst pin\n");
        rst_pin |= PIN_ENCRYPT;
        mt_set_gpio_mode(rst_pin, GPIO_MODE_00);
        mt_set_gpio_dir(rst_pin, GPIO_DIR_OUT);
        if (value)
            mt_set_gpio_out(rst_pin, GPIO_OUT_ONE);
        else
            mt_set_gpio_out(rst_pin, GPIO_OUT_ZERO);
    }
}

mtkfb {
    pinctrl-names = "lcd_bias_enp1_gpio", "lcd_bias_enp0_gpio",
        "lcd_bias_enn1_gpio", "lcd_bias_enn0_gpio",
        "lcm_rst_out1_gpio", "lcm_rst_out0_gpio",
        "mode_te_te";
    pinctrl-0 = <mtkfb_pins_lcd_bias_enp1>;
    pinctrl-1 = <mtkfb_pins_lcd_bias_enp0>;
    pinctrl-2 = <mtkfb_pins_lcd_bias_enn1>;
    pinctrl-3 = <mtkfb_pins_lcd_bias_enn0>;
    pinctrl-4 = <mtkfb_pins_lcm_rst_out1_gpio>;
    pinctrl-5 = <mtkfb_pins_lcm_rst_out0_gpio>;
    pinctrl-6 = <mtkfb_pins_lcm_dsi_te>;
    status = "okay";

    mtkfb_pins_lcm_rst_out0_gpio: lcm_rst_out0_gpio {
        pins_cmd_dat {
            pinmux = <PINMUX_GPIO42_FUNC_GPIO42>;
            slew-rate = <1>;
            output-low;
        };
    };
};

```

- Step 3: Implements the LCM init power function (must set)
 - According to the init process specified in the LCM datasheet, pull down/up the reset pin, delay, and set the LCM init register

```

static void lcm_init_power(void)
{
    SET RESET PIN(0);
    MDELAY(100);
    if (lcm_util.set_gpio_lcd_enp_bias) {
        lcm_util.set_gpio_lcd_enp_bias(1);
        i2c_w8(0x06, 0, 0x3e, 0x00, 0x0f);
        i2c_w8(0x06, 0, 0x3e, 0x01, 0x0f);

        LCM_LOGI("[zsq] -----lcm_init_power-----\n");
    }
    else
        LCM_LOGI("[zsq] set_gpio_lcd_enp_bias not defined\n");
}

```

- Step 4: Implements the LCM init function (must set)

- According to the init process specified in the LCM datasheet, pull down/up the reset pin, delay, and set the LCM init register:

```

4:
5: static void lcm_init(void *fdt)
6: {
7:     dprintf(0, "%s +++\n", __func__);
8:     SET_RESET_PIN(1, fdt);
9:     MDELAY(5);
10:    SET_RESET_PIN(0, fdt);
11:    MDELAY(15);
12:    SET_RESET_PIN(1, fdt);
13:    MDELAY(60);
14:
15:    is_send_data = false;
16:    push_table_by_type(lcm_bkl_setting,
17:        sizeof(lcm_bkl_setting) / sizeof(struct cust_setting_table),
18:        1);
19:    #if LCM_USED_2PORT_4SLICE
20:    push_table_by_type(lcm_pps_setting,
21:        sizeof(lcm_pps_setting) / sizeof(struct cust_setting_table),
22:        1);
23:    MDELAY(20);
24:    push_table_by_type(lcm_4slice_setting,
25:        sizeof(lcm_4slice_setting) / sizeof(struct cust_setting_table),
26:        1);
27:    MDELAY(20);
28:    #endif
29:    push_table(lcm_initialization_setting,
30:        sizeof(lcm_initialization_setting) / sizeof(struct LCM_setting_table),
31:        1);
32:
33:    dprintf(0, "%s ---\n", __func__);
34: } ? end lcm_init ?
35:

```

- Step 5: Implements LCM suspend/resume functions (optional)

```

static void lcm_suspend(void)
{
    int ret;

    push_table(lcm_suspend_setting,
        sizeof(lcm_suspend_setting) / sizeof(struct LCM_setting_table), 1);

    SET_RESET_PIN(0);
    MDELAY(10);
}

static void lcm_resume(void)
{
    lcm_init();
}

```

- Step 6: LCM compare function porting (if multi-panel is compatible, need LCM Compare ID function)

```

#define LCM_ID_NT36672C_JDI (0x41)

static unsigned int lcm_compare_id(void)
{
    unsigned int id = 0;
    unsigned char buffer[2];
    unsigned int array[16];

    LCM_LOGI("%S: enter\n", __func__);

    SET_RESET_PIN(1);
    MDELAY(10);
    SET_RESET_PIN(0);
    MDELAY(10);
    SET_RESET_PIN(1);
    MDELAY(10);

    array[0] = 0x00013700; /* read id return two byte, version and id */
    dsi_set_cmdq(array, 1, 1);

    read_reg_v2(0xDA, buffer, 1);
    id = buffer[0]; /* we only need ID */

    LCM_LOGI("%s, nt3672c_id_jdi=0x%08x\n", __func__, id);

    if (id == LCM_ID_NT36672C_JDI)
        return 1;
    else
        return 0;
} ? end lcm_compare_id ?

```

- Step 7: LCM set backlight function porting (only need config when LCM PWM used)

```
static void lcm_setbacklight(unsigned int level)
{
    LCM_LOGI("%s, nt35695 backlight: level = %d\n", __func__, level);
    bl_level[0].para_list[1] = level;
    push_table(bl_level, sizeof(bl_level) / sizeof(struct LCM_setting_table), 1);
}
```

- Step 8: Fills in the initialization parameters (must set)

```
static struct LCM_setting_table lcm_suspend_setting[] = {
    {0x28, 0, {} },
    {0x10, 0, {} },
    {REGFLAG_DELAY, 120, {} },
    {0x4F, 1, {0x01} },
    {REGFLAG_DELAY, 120, {} }
};

static struct LCM_setting_table init_setting[] = {
    {0xFF, 1, {0x24} },
    {0xFB, 1, {0x01} },
    {0x2D, 1, {0x08} },
    {0xFF, 1, {0x24} }, /* Return To CMD1 */
    {0x6E, 1, {0x10} }, /* Return To CMD1 */
    {0xFB, 1, {0x01} }, /* Return To CMD1 */
    {0xFF, 1, {0x10} }, /* Return To CMD1 */

    {0xFF, 1, {0x10} }, /* Return To CMD1 */
    {REGFLAG_UDELAY, 1, {} },
    #if (LCM_DSI_CMD_MODE)
    {0xBB, 1, {0x10} }, /*CMD MODE*/
    #else
    {0xBB, 1, {0x03} }, /*VDO MODE*/
    #endif
    {0x3B, 5, {0x03, 0x0A, 0x0A, 0x0A, 0x0A} },
    {0x53, 1, {0x24} },
    {0x55, 1, {0x00} },
    {0x5E, 1, {0x00} },
    {0xFF, 1, {0x24} }, /* CMD2 Page4 Entrance */
    {REGFLAG_UDELAY, 1, {} },
    {0xFB, 1, {0x01} },
}
```

- Step 9: Other used settings (copy from other drivers, must set)

```

#define GPIO_LED_BIAS_EN GPIO87
#define GPIO_LCD_RESET GPIO85
#define GPIO_LCM_ID0 GPIO109
#define GPIO_LCM_ID1 GPIO98
#define GPIO_LCD_BIAS_ENP GPIO82
#define GPIO_LCD_BIAS_ENN GPIO81
static const unsigned int BL_MIN_LEVEL = 20;
static LCM_UTIL_FUNCS lcm_util;
#define BIAS_I2C_ADDRESS (0x11)

#define SET_RESET_PIN(v, fdt) (lcm_util.set_reset_pin(v, fdt))
#define MDELAY(n) (lcm_util.mdelay(n))
#define UDELAY(n) (lcm_util.udelay(n))

/* ----- */
/* Local Functions */
/* ----- */

#define dsi_set_cmdq_V2(cmd, count, ppara, force_update) \
    lcm_util.dsi_set_cmdq_V2(cmd, count, ppara, force_update)
#define dsi_set_cmdq(pdata, queue_size, force_update) \
    lcm_util.dsi_set_cmdq(pdata, queue_size, force_update)
#define write_cmd(cmd) lcm_util.dsi_write_cmd(cmd)
#define write_regs(addr, pdata, byte_nums) \
    lcm_util.dsi_write_regs(addr, pdata, byte_nums)
#define read_reg(cmd) \
    lcm_util.dsi_dcs_read_lcm_reg(cmd)
#define read_reg_v2(cmd, buffer, buffer_size) \
    lcm_util.dsi_dcs_read_lcm_reg_v2(cmd, buffer, buffer_size)

/* ----- */
/* Local Constants */
/* ----- */

#define LCM_DSI_CMD_MODE 0
#define FRAME_WIDTH (1200)
#define FRAME_HEIGHT (2000)

#define REGFLAG_DELAY 0xFFFC
#define REGFLAG_UELAY 0xFFB
#define REGFLAG_END_OF_TABLE 0xFFD
#define REGFLAG_RESET_LOW 0xFFE
#define REGFLAG_RESET_HIGH 0xFFFF

static void lcm_set_gpio_output(unsigned int GPIO, unsigned int output)
{
    mt_set_gpio_mode(GPIO, GPIO_MODE_00);
    mt_set_gpio_dir(GPIO, GPIO_DIR_OUT);
    mt_set_gpio_out(GPIO, (output > 0) ? GPIO_OUT_ONE : GPIO_OUT_ZERO);
}

struct LCM_setting_table {
    unsigned int cmd;
    unsigned char count;
    unsigned char para_list[200];
};

static void push_table(struct LCM_setting_table *table, unsigned int count,
    unsigned char force_update)
{
    unsigned int i;
    unsigned int cmd;
    for (i = 0; i < count; i++) {
        cmd = table[i].cmd;
        switch (cmd) {
            case REGFLAG_DELAY:
                if (table[i].count <= 10)
                    MDELAY(table[i].count);
                else
                    MDELAY(table[i].count);
                break;
            case REGFLAG_UELAY:
                UDELAY(table[i].count);
                break;
            case REGFLAG_END_OF_TABLE:
                break;
            default:
                dsi_set_cmdq_V2(cmd, table[i].count, table[i].para_list,
                    force_update);
                break;
        }
    }
}

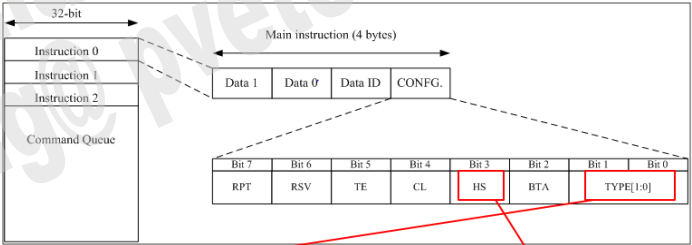
static void lcm_set_util_funcs(const LCM_UTIL_FUNCS *util)
{
    memcpy(&lcm_util, util, sizeof(LCM_UTIL_FUNCS));
}

```

5.3 DSI Command Usage in LK

DSI Command Queue(1/2)

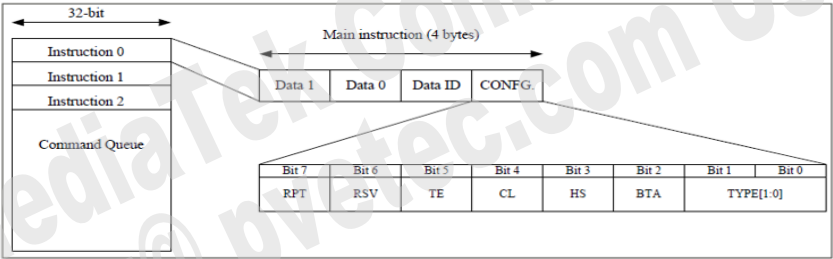
- Two dedicated command queues with 32-bit wide and 32-entry depth for each.



- Type[1:0]
 - 2'b00: (short packet) Read/write command
 - 2'b01: (long packet) Frame buffer write command (from LCD)
 - 2'b10: (long packet) Generic long packet write command (from command queue)
 - 2'b11: (short packet) Frame buffer read command



DSI Command Queue(2/2)



Byte 3	Byte 2	Byte 1	Byte 0
Data 1	Data 0	Data ID	CONFIG.

Fig. 5-8: Type-0 instruction format

Byte 3	Byte 2	Byte 1	Byte 0
Mem start 1 (optional)	Mem start 0	Data ID	CONFIG.

Fig. 5-9: Type-1 instruction format

Byte 3	Byte 2	Byte 1	Byte 0
WC 1	WC 0	Data ID	CONFIG.
Data 3	Data 2	Data 1	Data 0
		Data WC-1	Data WC-2

Fig. 5-10: Type-2 instruction format

Byte 3	Byte 2	Byte 1	Byte 0
Mem start 1 (optional)	Mem start 0	Data ID	CONFIG.

Fig. 5-11: Type-3 instruction format

dsi_set_cmdq(*pdata,queue_size,force_update)

Type 0 & Type 2

```
static void init_lcm_registers(void)
```

```
{
    unsigned int data_array[16];
    data_array[0] = 0x00043902;
    data_array[1] = 0x6983FFB9;
    dsi_set_cmdq(&data_array, 2, 1);
```

1.data_array[0]=0x00043902
0x02→type 2 Generic Long Write
0x39→DI=0x39,DCS long write
command(long packet)
0x0004→VWC=4,4Byte data
2.data_array[1]=0x6983FFB9写LCM
Register
→command: 0xB9
params: 0xFF, 0x83, 0x69

```
data_array[0] = 0x073A1500;
dsi_set_cmdq(&data_array, 1, 1);
```

1.data_array[0]=0x073A1500
0x00→type 0 Short Packet Write
0x15→DI=0x15,DCS short write, 1
params
0x05→DI=0x05,DCS short write, no
params

```
data_array[0] = 0x00110500; // Sleep Out
dsi_set_cmdq(&data_array, 1, 1);
MDELAY(100);
```

```
data_array[0] = 0x00290500; // Display On
dsi_set_cmdq(&data_array, 1, 1);
```

dsi_set_cmdq_V2(cmd, count, *para_list, force_update)

```
static void lcm_resume(void)
```

```
{
    push_table(lcm_sleep_out_setting, sizeof(lcm_sleep_out_setting) / sizeof(struct LCM_setting_table), 1);
}
```

```
static struct LCM_setting_table lcm_sleep_out_setting[] = {
    // Sleep Out
    {0x11, 1, {0x00}},
    {REGFLAG_DELAY, 120, {}},
    // Display ON
    {0x29, 1, {0x00}},
    {REGFLAG_END_OF_TABLE, 0x00, {}}
};
```

```
struct LCM_setting_table {
    unsigned cmd;
    unsigned char count;
    unsigned char para_list[64];
};
```

```
void push_table(struct LCM_setting_table *table, unsigned int count, unsigned char force_update)
```

```
{
    unsigned int i;
    for(i = 0; i < count; i++) {
        unsigned cmd;
        cmd = table[i].cmd;
        switch (cmd) {
            case REGFLAG_DELAY :
                MDELAY(table[i].count);
                break;
            case REGFLAG_END_OF_TABLE :
                break;
            default:
                dsi_set_cmdq_V2(cmd, table[i].count, table[i].para_list, force_update);
        }
    }
}
```

dsi_set_cmdq_V2(cmd, count, *para_list, force_update)

```
static void lcm_init(void)
```

```
{
    SET_RESET_PIN(1);
    SET_RESET_PIN(0);
    MDELAY(1);
    SET_RESET_PIN(1);
    MDELAY(10);
    push_table(lcm_initialization_setting, sizeof(lcm_initialization_setting) / sizeof(struct LCM_setting_table), 1);
}
```

```
static struct LCM_setting_table lcm_initialization_setting[] = {
```

```
/*
Note :
```

Data ID will depends on the following rule.

```
count of parameters > 1    => Data ID = 0x39
count of parameters = 1    => Data ID = 0x15
count of parameters = 0    => Data ID = 0x05
```

Structure Format :

```
{DCS command, count of parameters, {parameter list}}
{REGFLAG_DELAY, milliseconds of time, {}},
```

```
...
```

Setting ending by predefined flag

```
{REGFLAG_END_OF_TABLE, 0x00, {}}
```

```
{0xB9, 3, {0xFF, 0x83, 0x69}},
{REGFLAG_DELAY, 10, {}},
{0xB0, 2, {0x01, 0x0B}},
{REGFLAG_DELAY, 10, {}},
{0xB2, 15, {0x00, 0x20, 0x05, 0x05,
             0x70, 0x00, 0xFF, 0x00,
             0x00, 0x00, 0x00, 0x03,
             0x03, 0x00, 0x01}},
{REGFLAG_DELAY, 10, {}},
```

6 Kernel DRM Drivers

- **Key path:** /kernel/ kernel_device_modules-6.1/drivers/gpu/drm/
- **MTK key path:** /kernel/ kernel_device_modules-6.1/drivers/gpu/drm/mediatek/Mediatek_v2

Name	Files
DRM framework relative	mtk_drm_drv.c, mtk_drm_crtc.c, mtk_drm_fb.c, mtk_drm_gem.c, mtk_drm_plane.c...
MTK DDP	mtk_disp_ccorr.c , mtk_disp_gamma.c, mtk_disp_dither.c, mtk_disp_ovl.c, mtk_disp_rdma.c, mtk_disp_dsi.c , mtk_disp_wdma.c ...
Connector	mtk_dsi.c, mtk_dpi.c, mtk_hdmi.c, mtk_mipi_tx.c...
Name	Path
DRM Core	drivers/gpu/drm/
MTK implement	drivers/gpu/drm/mediatek/mediatek_v2 (K510) OR drivers/gpu/drm/mediatek/ (K419)
Panel driver	drivers/gpu/drm/panel/
DRM Header File	include/drm/ include/uapi/drm/

6.1 Add Kernel LCM Driver in Kernel

- Step 1: Adds your <lcmm driver>
- Step 2: Adds your <lcmm msg> to Kconfig & Makefile
- Step 3: Checks ProjectConfig.
 - Adds your DRM panel configuration and set it to 'm'

```
arch/arm64/configs/mgk_64_k61_defconfig |
CONFIG_MTK_PANEL_PROTECT=m
CONFIG_DRM_PANEL_BOE_JD9365DA=m
CONFIG_DRM_PANEL_SERDES=m
CONFIG_DRM_PANEL_TRULY_TD4330_VDO=m
CONFIG_DRM_PANEL_TRULY_TD4330_CMD=m
```

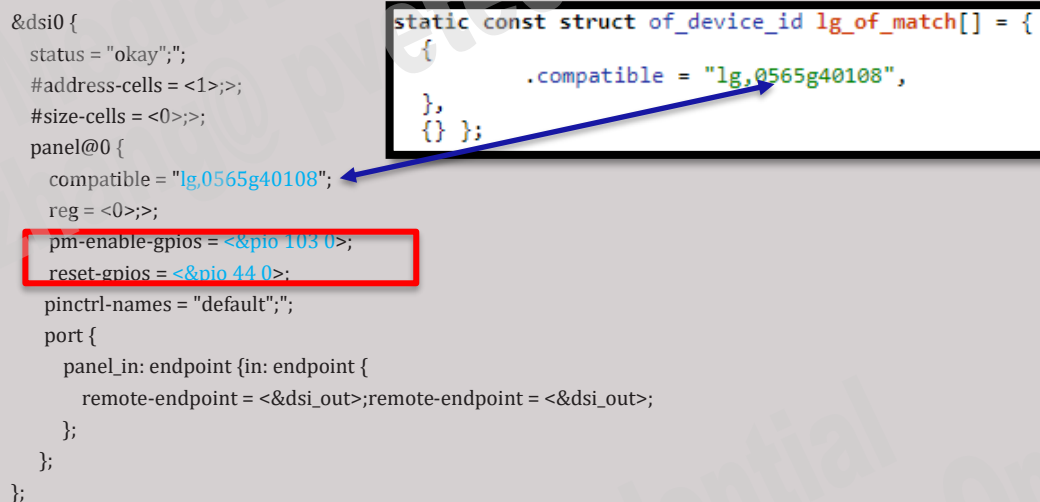
- Adds below GKI config to device/mediatekxxx/\$ (project) /ko_order_table.csv

Name	Date modified	Type
init-project.rc	1/5/2023 3:35 PM	RC F
ko_order_table.csv	1/5/2023 3:35 PM	Mic
mtk_panel_ext.ko, /drivers/gpu/drm/mediatek/mediatek_v2/mtk_panel_ext.ko, ramdisk, Y, Y, user/userdebug/eng		
panel-himax-hx83121a-vdo.ko, /drivers/gpu/drm/panel/panel-himax-hx83121a-vdo.ko, ramdisk, Y, Y, user/userdebug/eng		

- Adds ko to kleaf kernel/kernel_device_modules-6.1/kernel/kleaf/

```
kernel/kleaf/mgk_64_k61_bzl
"drivers/gpu/drm/mediatek/mml/mtk-mml.ko",
"drivers/gpu/drm/panel/bridge-serdes-max96789.ko",
"drivers/gpu/drm/panel/k6985v1_64_alpha/panel-nt37705-alpha-cmd.ko",
"drivers/gpu/drm/panel/k6989v1_64_alpha/panel-ili7838e-alpha-cmd.ko",
"drivers/gpu/drm/panel/k6989v1_64_alpha/panel-ili7838e-dv2-alpha-cmd.ko"
```

- Step 4: Set the LCM node to the project DTS.
 - /kernel/ kernel_device_modules-6.1/arch/arm64/boot/dts/mediatek/\$ (project) .dts
 - → Sets panel name must be the same as of_device_id in your DRM driver
 - Checks GPIO pins → lcm_reset, bl_en, avdd_en, avee_en & other control pins



```

&dsi0 {
    status = "okay";
    #address-cells = <1>;
    #size-cells = <0>;
    panel@0 {
        compatible = "lg,0565g40108";
        reg = <0>;
        pm-enable-gpios = <&pio 103 0>;
        reset-gpios = <&pio 44 0>;
        pinctrl-names = "default";
        port {
            panel_in: endpoint {in: endpoint {
                remote-endpoint = <&dsi_out>;remote-endpoint = <&dsi_out>;
            };
        };
    };
};
  
```

```

static const struct of_device_id lg_of_match[] = {
    {
        .compatible = "lg,0565g40108",
    },
    {}
};
  
```

- Adds you panel dts node, from 0 to N, and you must follow the following rules.

Kernel node must set panel 0~X, refer to lk
 Config CUSTOM_LK_LCM = "LCM_B,LCM_A".
 So dts note need to set as:

```

panel@0{
    Compatible = "LCM_B"
};
panel1@1{
    Compatible = "LCM_A"
};
  
```

CUSTOM_LK_LCM= "hx83112b_fhdp_dsi_vdo_auo_rt4801 lg_fhdp_dsi_cmd"

```

panel@0 {
    compatible = "hx83112b_fhdp_dsi_cmd_auo_rt4801_drv";
    reg = <0>;
    pm-enable-gpios = <&pio 103 0>;
    reset-gpios = <&pio 44 0>;
    pinctrl-names = "default";
    port {
        panel_in: endpoint {in: endpoint {
            remote-endpoint = <&dsi_out>;remote-endpoint =
            <&dsi_out>;
        };
    };
};
panel1@1 {
    compatible = "lg_fhdp_dsi_cmd";
    reg = <0>;
    pm-enable-gpios = <&pio 103 0>;
    reset-gpios = <&pio 44 0>;
    pinctrl-names = "default";
    port {
        panel_in1: endpoint {
            remote-endpoint = <&dsi_out>;remote-endpoint =
            <&dsi_out>;
        };
    };
};
  
```

- Single port setting

```

1: static struct mtk_panel_params ext_params = {
2:     .change_fps_by_vfp_send_cmd = 1,
3:     .vfp_low_power = 198,
4:     .cust_esd_check = 0,
5:     .esd_check_enable = 1,
6:     .ssc_enable = 0,
7:     .lane_swap_en = 0,
8:     .output_mode = MTK_PANEL_DSC_SINGLE_PORT,
9: };
10:
11: &dsi0 {
12:     status = "okay";
13:     #address-cells = <1>;
14:     #size-cells = <0>;
15:     panel1@0 {
16:         compatible = "boe,hx83102j,vdo,120hz";
17:         reg = <0>;
18:         pm-enable-gpios = <&pio 87 0>;
19:         reset-gpios = <&pio 85 0>;
20:         bias-gpios = <&pio 82 0>,
21:             <&pio 81 0>;
22:         pinctrl-names = "default";
23:         port {
24:             panel_in1: endpoint {
25:                 remote-endpoint = <&dsi_out>;
26:             };
27:         };
28:     };
29:     ports {
30:         port {
31:             dsi_out: endpoint {
32:                 remote-endpoint = <&panel_in1>;
33:             };
34:         };
35:     };
36: };

```

- Dual port setting

```

55: static struct mtk_panel_params ext_params = {
56:     .pll_clk = PLL_CLOCK,
57:     .cust_esd_check = 0,
58:     .esd_check_enable = 1,
59:     .ssc_enable = 0,
60:     .output_mode = MTK_PANEL_DUAL_PORT,
61:     .lcm_cmd_if = MTK_PANEL_DUAL_PORT,
62:     .lane_swap_en = 1
63: };
64:
65: &dsi0 {
66:     status = "okay";
67:     #address-cells = <1>;
68:     #size-cells = <0>;
69:     panel1@0 {
70:         compatible = "edo,hx5329,cmd,120hz";
71:         reg = <0>;
72:         reset-gpios = <&pio 86 0>;
73:         switch_en-gpios = <&pio 68 0>;
74:         tp_lv8_en-gpios = <&pio 84 0>;
75:         mt6373_iol1-gpios = <&mt6373_pio 11 0>;
76:         pinctrl-names = "panel_reset_en_gpio", "panel_switch_en_gpio",
77:             "panel_power_en";
78:         pinctrl-0 = <&disp_panel_reset_en>;
79:         pinctrl-1 = <&disp_panel_switch_en>;
80:         pinctrl-2 = <&disp_panel_power_en>;
81:         port {
82:             panel_in: endpoint {
83:                 remote-endpoint = <&dsi_out>;
84:             };
85:         };
86:     };
87:     ports {
88:         port@0 {
89:             reg = <0>;
90:             dsi_out: endpoint {
91:                 remote-endpoint = <&panel_in>;
92:             };
93:         };
94:         port@1 {
95:             reg = <1>;
96:             master_out: endpoint {
97:                 remote-endpoint = <&slave_in>;
98:             };
99:         };
100:     };
101: };
102:
103: &dsi1 {
104:     mediatek,dual-dsi-slave;
105:     status = "okay";
106:     ports {
107:         port@1 {
108:             reg = <1>;
109:             slave_in: endpoint {
110:                 remote-endpoint = <&master_out>;
111:             };
112:         };
113:     };
114: };

```

6.2 Porting LCM Driver in Kernel

- Panel driver structure brief introduction (must set)
 - DRM original structure

```
static const struct drm_display_mode default_mode = {
    .clock = 422163,
    .hdisplay = 1080,
    .hsync_start = 1080 + 274,
    .hsync_end = 1080 + 274 + 12,
    .htotal = 1080 + 274 + 12 + 56,
    .vdisplay = 2400,
    .vsync_start = 2400 + 2528,
    .vsync_end = 2400 + 2528 + 10,
    .vtotal = 2400 + 2528 + 10 + 10,
    .vrefresh = 60,
};
```

- MTK extension structure

```
struct mtk_panel_params {
    unsigned int pll_clk;
    unsigned int data_rate;
    struct mtk_dsi_phy_timcon phy_timcon;
    unsigned int vfp_low_power;
    struct dynamic_mipi_params dyn;
    struct dynamic_fps_params dyn_fps;
    unsigned int cust_esd_check;
    unsigned int esd_check_enable;
    struct esd_check_item lcm_esd_check_table[ESD_CHECK_NUM];
    unsigned int ssc_disable;
    unsigned int ssc_range;
    int lcm_color_mode;
    unsigned int min_luminance;
    unsigned int average_luminance;
    unsigned int max_luminance;
    unsigned int round_corner_en;
    unsigned int corner_pattern_height;
    unsigned int corner_pattern_height_bot;
    unsigned int corner_pattern_tp_size;
    void *corner_pattern_lt_addr;
    unsigned int physical_width_um;
    unsigned int physical_height_um;
    unsigned int lane_swap_en;
    unsigned int is_cphy;
    enum MIPITX_PHY_LANE_SWAP
        lane_swap[MIPITX_PHY_PORT_NUM][MIPITX_PHY_LANE_NUM];
    struct mtk_panel_dsc_params dsc_params;
    unsigned int output_mode;
    unsigned int hbm_en_time;
    unsigned int hbm_dis_time;
    unsigned int lcm_index;
};
```

- MTK extension structure (DSC)

```

struct mtk_panel_dsc_params {
    unsigned int enable;
    unsigned int dual_dsc_enable;
    unsigned int bdg_dsc_enable;
    unsigned int ver; /* [7:4] major [3:0] minor */
    unsigned int slice_mode;
    unsigned int rgb_swap;
    unsigned int dsc_cfg;
    unsigned int rct_on;
    unsigned int bit_per_channel;
    unsigned int dsc_line_buf_depth;
    unsigned int bp_enable;
    unsigned int bit_per_pixel;
    unsigned int pic_height; /* need to check */
    unsigned int pic_width; /* need to check */
    unsigned int slice_height;
    unsigned int slice_width;
    unsigned int chunk_size;
    unsigned int xmit_delay;
    unsigned int dec_delay;
    unsigned int scale_value;
    unsigned int increment_interval;
    unsigned int decrement_interval;
    unsigned int line_bpg_offset;
    unsigned int nfl_bpg_offset;
    unsigned int slice_bpg_offset;
    unsigned int initial_offset;
    unsigned int final_offset;
    unsigned int flatness_minqp;
    unsigned int flatness_maxqp;
    unsigned int rc_model_size;
    unsigned int rc_edge_factor;
    unsigned int rc_quant_incr_limit0;
    unsigned int rc_quant_incr_limit1;
    unsigned int rc_tgt_offset_hi;
    unsigned int rc_tgt_offset_lo;
    struct mtk_panel_dsc_ext_pps_cfg ext_pps_cfg;
} __packed __attribute__((aligned(4)));

```

- Panel driver function's brief introduction
 - DRM original function

```

static const struct drm_panel_funcs mtk_drm_funcs = {
    struct mtk_panel_funcs {
        int (*set_backlight_cmdq) (void *dsi_drv, dcs_write_gce cb, void *handle, unsigned int level);
        int (*set_backlight_grp_cmdq) (void *dsi_drv, dcs_grp_write_gce cb, void *handle, unsigned int level);
        int (*reset) (struct drm_panel *panel, int on);
        int (*ata_check) (struct drm_panel *panel);
        int (*ext_param_set) (struct drm_panel *panel, unsigned int mode);
        int (*ext_param_get) (struct mtk_panel_params *ext_para, unsigned int mode);
        int (*mode_switch) (struct drm_panel *panel, unsigned int cur_mode, unsigned int dst_mode, enum MTK_PANEL_MODE_SWITCH_STAGE stage);
        int (*get_virtual_height) (void);
        int (*get_virtual_width) (void);
        int (*doze_enable_start) (struct drm_panel *panel, void *dsi_drv, dcs_write_gce cb, void *handle);
        int (*doze_enable) (struct drm_panel *panel, void *dsi_drv, dcs_write_gce cb, void *handle);
        int (*doze_disable) (struct drm_panel *panel, void *dsi_drv, dcs_write_gce cb, void *handle);
        int (*doze_post_disp_on) (struct drm_panel *panel);
        int (*doze_area) (struct drm_panel *panel, void *dsi_drv, dcs_write_gce cb, void *handle);
        unsigned long (*doze_get_mode_flags) (struct drm_panel *panel, int aod_en);
        int (*hbm_set_cmdq) (struct drm_panel *panel, void *dsi_drv, dcs_write_gce cb, void *handle, bool en);
        void (*hbm_get_state) (struct drm_panel *panel, bool *state);
        void (*hbm_get_wait_state) (struct drm_panel *panel, bool *wait);
        bool (*hbm_set_wait_state) (struct drm_panel *panel, bool wait);
    };

```

Panel power disable

sequence

on

sequence

configuration

- DRM extension function

- Porting the probe function (must set)
- Porting the remove function (must set)

```
static void lcm_remove(struct mipi_dsi_device *dsi)
{
    struct lcm *ctx = mipi_dsi_get_drvdata(dsi);
    #if defined(CONFIG_MTK_PANEL_EXT)
    struct mtk_panel_ctx *ext_ctx = find_panel_ctx(&ctx->panel);
    #endif

    mipi_dsi_detach(dsi);
    drm_panel_remove(&ctx->panel);
    #if defined(CONFIG_MTK_PANEL_EXT)
    mtk_panel_detach(ext_ctx);
    mtk_panel_remove(ext_ctx);
    #endif
}
```

- Porting prepare function (must set)

```
static int lcm_prepare(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);

    pr_info("%s +\n", __func__);

    lcm_poweron(ctx);
    lcm_panel_init(ctx);
    serdes_pre_enable(ctx->bridge);

    #if defined(CONFIG_MTK_PANEL_EXT)
    mtk_panel_tch_rst(panel);
    #endif
    pr_info("%s -\n", __func__);

    return 0;
}

static void lcm_panel_init(struct lcm *ctx)
{
    pr_info("%s +\n", __func__);
    pr_info("%s -\n", __func__);
}

if (remote_node != dev->of_node) {
    pr_info("%s+ skip probe due to not current lcm\n", __func__);
    return -ENODEV;
}
```

- Port the panel_init function (must set)
- Porting enable function (must set)
- Porting disable function (must set)

```
static int lcm_enable(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);

    pr_info("%s +\n", __func__);

    serdes_enable(ctx->bridge);
    pr_info("%s -\n", __func__);

    return 0;
}
```

```
static int lcm_disable(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);

    pr_info("%s +\n", __func__);

    serdes_disable(ctx->bridge);

    pr_info("%s -\n", __func__);

    return 0;
}
```

- Porting the unprepare function (must set)

```
static int lcm_unprepare(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);

    pr_info("%s +\n", __func__);

    lcm_poweroff(ctx);

    pr_info("%s -\n", __func__);

    return 0;
}
```

- Port the get_modes function (must set)

```
static int lcm_get_modes(struct drm_panel *panel,
    struct drm_connector *connector)
{
    struct drm_display_mode *mode;

    pr_info("%s +\n", __func__);
    mode = drm_mode_duplicate(connector->dev, &default_mode);
    if (!mode) {
        pr_info("failed to add mode %ux%ux@%u\n",
            default_mode.hdisplay,
            default_mode.vdisplay,
            drm_mode_vrefresh(&default_mode));
        return -ENOMEM;
    }

    mode->type = DRM_MODE_TYPE_DRIVER | DRM_MODE_TYPE_PREFERRED;

    drm_mode_set_name(mode);
    drm_mode_probed_add(connector, mode);

    connector->display_info.width_mm = default_mode.width_mm;
    connector->display_info.height_mm = default_mode.height_mm;

    pr_info("%s -\n", __func__);

    return 1;
} « end lcm_get_modes »
```

- Porting MTK extension reset function (optional)
- Porting MTK extension set_backlight_cmdq function (by case)
- Port MTK extension ext_param_set function (by case)
- Porting MTK extension: ata_check function (optional)

- Port the MTK extension parameters structure (must set)

```
static struct mtk_panel_params ext_params = {  
    .pll_clk = 369,  
    .data_rate = 738,  
    .vfp_low_power = 4178,  
    .cust_esd_check = 0,  
    .esd_check_enable = 1,  
    .lcm_esd_check_table[0] = {  
        .cmd = 0x0A, .count = 1, .para_list[0] = 0x9C,  
    },  
    .is_cphy = 1,  
};
```

Notes:

- In super frame mode, the pll_clk/crop_width/crop_height/physical_width/physical_height must be set, otherwise, the pll_clk must be set
 - Pll_clk: mipi clock
 - Crop width/crop height: In super frame mode, it should be set to correspond to the width/height of one of the panels
 - Physical_width/physical_height: In super frame mode, physical_with should be set to the sum of the widths of the two panels. And the physical_height should be set to the larger of the two panels

7 Extremely precise FPS

- In pen case, very precise FPS (120+-0.01) is required, which can be achieved through data_rate_khz
- First, calculate the approximate value (value of “ideal bit freq”) through the table (ask PM for it) and fill it into data_rate, then fine-tune it through data_rate_khz
- Note: even if data_rate_khz is set, data_rate/pll_clk also needs to be set (whether it's LK or Kernel)
- Data_rate_khz needs to be fine-tuned based on the measurement of FPS, and currently cannot be calculated theoretically.
- Vdo_per_frame_lp_enable corresponds to the “video mode keep hs mode” in the table below, which has a relatively large impact on the FPS of vdo mode

Panel Size		Vertical (unit: line)				Horizontal (unit: pixel)				Q-PHY		Compression Ratio	24:16:8 RGB	range: (DPS)	Bit Rate (Q-PHY)		
H-Size (mm)	V-Size (mm)	VACT	VTP	VFP	VBP	HACT	HTP	HSA	HBP	C-Q-PHY	bits_per_line	QRC	FrameRate (Hz/line)	mode selection	ideal Bit/line		
5.22	2540	2440	60	0	12	1080	60	12	80	0	4	0.3333	24	3	900		
Option										PHY Timing Spec (unit: pixel)		PHY Timing Spec (unit: US)		Unit: Frame per Second(FPS)	Line Time Unit: us	Frame Time Unit: ms	Drop Clock
HS-BLUF_PVS	Video Mode Keep HS Mode	Command Mode Keep HS Mode	Null Packet Word Count Command Mode	PROTOCOL_DEV_EN	Per-Frame Post Processing	Display Pipe	HEP	HSA	HBP	CS-prealign	CS-post	Real Frame Rate (FPS)	Real LineTime	Real FrameTime	Drop Clock		
00	N	Y	mode w/ Null Packet	03	0	0	1	547.33	Don't Care	52.67	0	0	332.3622	3.846	9.769	900.00	

```
00855: .data_rate_khz = 960298,
00856: .data_rate = 960,
```

```
00398: //
00399: struct mtk_panel_params {
00400:     unsigned int change_fps_by_vfp_send_cmd;
00401:     unsigned int pll_clk;
00402:     unsigned int data_rate;
00403:     //either pll_clk or data_rate must be set, event if data_rate_khz is set
00404:     unsigned int data_rate_khz; //only used in exact value for fps
00405:     // Enable video mode per frame lp */
```

8 Bring up Debug SOP

8.1 Panel Not Work in LK Stage

- Does backlight work ?
 - If yes, check the next step.
 - If not, is the backlight setting correct? (The backlight uses the DTS setting even when it is in LK)
 - Can override this by pulling the backlight control GPIO to high when bringing it up
 - If backlight control is done by I2C, seek help from the I2C vendor.

- disp_pwm mode

```
pwmleds: pwmleds {
    compatible = "mediatek,pwm-leds";
    status = "disabled";
    backlight {
        label = "lcd-backlight";
        pwms = <&disp_pwm0 0 500000>;
        max-brightness = <255>;
        pwm-names = "lcd-backlight";
    };
};
```

- I2C mode

```
mtk_leds: mtk_leds {
    backlight {
        label = "lcd-backlight";
        max-brightness = <255>;
        max-hw-brightness = <2047>;
    };
};

&mtk_leds {
    compatible = "mediatek,i2c-leds";
    backlight {
        led_mode = <6>;
        gate_enable = <1>;
        pwm_config = <0 1 0 0 0>;
    };
};
```

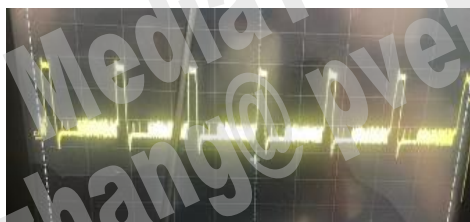
- Lcm pwm mode

```
1:
2: &mtk_leds {
3:     compatible = "mediatek,disp-leds";
4:
5:     backlight {
6:         led_mode = <4>;
7:         gate_enable = <1>;
8:         pwm_config = <0 1 0 0 0>;
9:     };
10: };
```

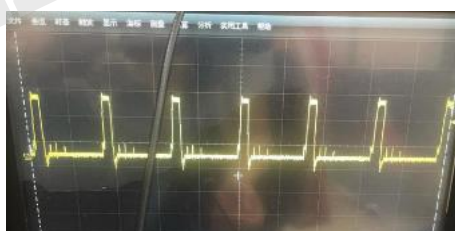
- Is the panel power supply correct?
 - If yes, check the next step
 - If not, measure the voltage of each path. If any one is abnormal, ask the PMIC owner how to control.
- Is the panel power-on sequence correct?
 - Measure the voltage with an oscilloscope.
- Does the Init command work? (Read DDIC register: 0x0A; if not 0, it means okay. If not okay, panel support is needed. Ask for the panel vendor.)
 - If yes, check the next step.
 - If not
 - Send the init command with generic or DCS.
 - Test the BIST pattern (obtain it from the panel vendor).
 - BIST can work, which means panel power control is correct.

- BIST cannot work
- Does the configuration need a lane swap?
- Use a logic analyzer to capture the data of sending the init command to the panel vendor for inspection.
- Does the MIPI signal meet the requirements? (CPHY needs to measure the signal of the pin at the panel side)

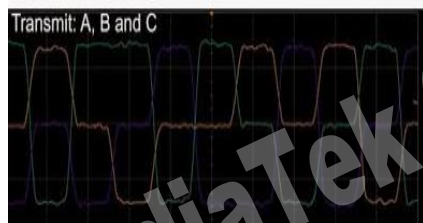
- Dphy wave with video data



- Dphy wave without VDO data



- Cphy wave



- Does the logo file exist? If there is no logo file, only a line of text is displayed on the screen, which needs to be carefully observed

8.2 Panel Does Work in the Kernel Stage

- Check the status of the panel power
 - At the Kernel stage, the regulator will disable when nobody is using it, and the panel driver needs to get and enable it.
- Check the backlight status
 - AAL may turn off the backlight when it malfunctions. Disable AAL and test again.
 - Does the backlight control function run?
- Check the status of the Kernel panel driver
 - Observe the log to see if the driver loads normally.
 - Is ESD enabled? Disable it and test again
 - Does multi-FPS support? Only enable one of them when bringing it up

8.3 Panel Does not Work in Resume Stage

- Check the status of the panel power
 - Check the status of the panel power-on code work

- Check the backlight status
 - Check the backlight control code's work status
- Check the status of the Kernel panel driver
 - Observe the log to see if the driver loads normally
 - Dump the register and compare it with LK

8.4 Panel Flicker

- Does DSC enable
 - Yes, check the DSC parameters
 - Does DSC test pattern work normal? (Used cmd: echo reg_write 0x14015078 0x400000FF > /proc/clkgdbg ; cat /proc/clkgdbg)
 - Yes, call for help from the display owner
 - No, is there a normal work time? If yes, dump and compare reg when work normal and wrong
 - Dump cmd:
 - a) Kernel stage: adb shell "echo mobile:on > /d/mtkfb";adb shell "echo diagnose>/d/mtkfb && cat /d/mtkfb" > mtkfb.txt and search "DSC"
 - b) LK stage: call the dump function at the end of the ddp_dsc_config
 - Typical DSC flicker:



- Similar flickering may be caused by AFBC, try turning off AFBC and test it again
 - Turn off AFBC command:

```
adb shell setprop debug.mediatek.disp_decompress 0
adb shell stop
adb shell start
```

- Typical AFBC flicker:

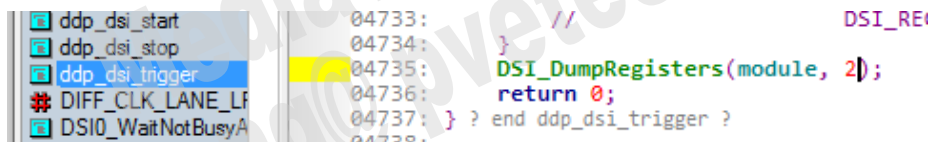


- Check the DSI test pattern work status
 - DSI test pattern cmd: `echo reg_write 0x140178 0x31 > /proc/clkgdb ; cat /proc/clkgdb`
 - 0x1401xxxx is dsi base addr, its value will vary depending on the soc
 - You can obtain it with the dsix node in the dts file
 - 0x178 is the DSI test pattern address, obtain it using SOC CODA (get CODA from CPM)
 - If yes, call for help from the display owner.
 - If not, does the log contain the phrase 'dsi underrun'?
 - Is there a normal work time
 - Yes, dumping and comparing work normally and incorrectly for DSI/MIPITX/DSC registers.
 - Dump cmd: `adb shell "echo mobile:on > /d/mtkfb";adb shell "echo diagnose>/d/mtkfb && cat /d/mtkfb"`
`> mtkfb.txt search "dsi" in it`
 - Does it only appear after a certain operation (such as switching FPS)?
 - Dump and compare the register when it works normally and when it is faulty.
 - For example (works incorrectly after changing fps):
 - Call for help from the DDIC vendor.
 - Is the order of changes in mmclk correct when change fps? (When mmclk increases, set mmclk first and then switch fps. when mmclk decreases, switch fps first and then set mmclk. refer to the `mtk_dsi_set_mmclk_by_datarate` function for the calculation method of mmclk, and refer to the `mtk_crtc_disp_mode_switch_begin` function for the switching logic)

9 Dump Reg

9.1 LK Stage

- Call DSI_DumpRegister() in end of Ddp_dsi.c/ddp_dsi_trigger()



```

04733: // DSI_RE
04734: }
04735: DSI_DumpRegisters(module, 2);
04736: return 0;
04737: } ? end ddp_dsi_trigger ?
04738:

```

- Call dsc_dump() in end of Ddp_dsi.c/dsc_config()



```

00306: DISP_REG_SET(handl
00307: DISP_REG_SET(handl
00308: }
00309: dsc_dump(module, 2);
00310: return 0;
00311: } ? end dsc config ?

```

- LK display log off by default in ddp_log.h, need turn on it frist

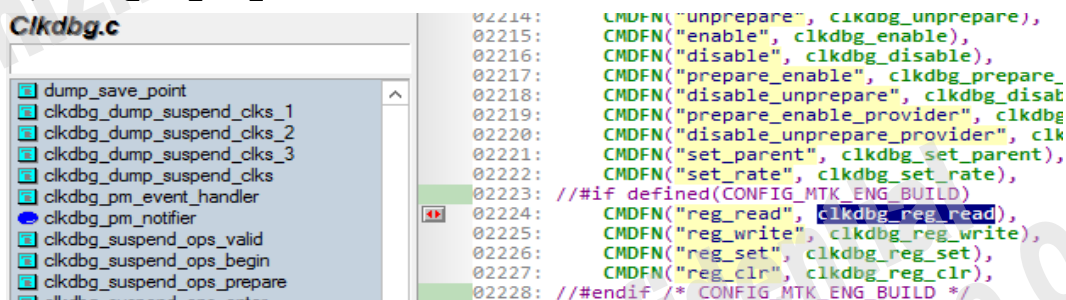
```

//
// #define DDP_INFO 0
// #define DDP_CRITICAL 0
// #define LOCAL_TRACE 1
//

```

9.2 Kernel Dump Reg

- Get dump information from the log
 - Use the following commands when the screen is shown and provide the mtkfb.txt file.
 - adb shell "echo mobile:on > /d/mtkfb"
 - adb shell "echo diagnose>/d/mtkfb && cat /d/mtkfb" > mtkfb.txt
- How to enable the command: clkdbg
 - File path: kernel-xx/drivers/clk/mediatek/clkdbg.c/common_cmds[]
 - Func: clkdbg_reg_read() and clkdbg_reg_write()
 - How: if clkdbg_reg_read() and clkdbg_reg_write() was enclosed within pre-defined directives #if defined(CONFIG_MTK_ENG_BUILD, mask it



```

02214: CMDFN("unprepare", clkdbg_unprepare),
02215: CMDFN("enable", clkdbg_enable),
02216: CMDFN("disable", clkdbg_disable),
02217: CMDFN("prepare_enable", clkdbg_prepare_enable),
02218: CMDFN("disable_unprepare", clkdbg_disable_unprepare),
02219: CMDFN("prepare_enable_provider", clkdbg_prepare_enable_provider),
02220: CMDFN("disable_unprepare_provider", clkdbg_disable_unprepare_provider),
02221: CMDFN("set_parent", clkdbg_set_parent),
02222: CMDFN("set_rate", clkdbg_set_rate),
02223: // #if defined(CONFIG_MTK_ENG_BUILD)
02224: CMDFN("reg_read", clkdbg_reg_read),
02225: CMDFN("reg_write", clkdbg_reg_write),
02226: CMDFN("reg_set", clkdbg_reg_set),
02227: CMDFN("reg_clr", clkdbg_reg_clr),
02228: // #endif /* CONFIG_MTK_ENG_BUILD */

```

9.3 Adb cmd Debug Reg Setting

- Command for reading/writing register
 - Read reg: echo reg_read addr> /proc/clkdbg ; cat /proc/clkdbg
 - Read reg with length: echo reg_read_len addr length> /proc/clkdbg ; cat /proc/clkdbg
 - Write reg: echo reg_write addr value> /proc/clkdbg ; cat /proc/clkdbg

- For example:
 - `echo reg_read 0x1400D178 > /proc/clkdbg ; cat /proc/clkdbg`
 - `echo reg_write 0x1400D178 0x31 > /proc/clkdbg ; cat /proc/clkdbg`
 - `echo reg_read_len 0x1400D000 0x00001000 > /proc/clkdbg ; cat /proc/clkdbg`

Notes:

- Usually, MTK supporter say let xxx's yyy be zzz, xxx means some module, get this module's base addr with Debug SOP, yyy means offset, zzz means set value
- The executed cmd will be: `echo reg_write xxx_base_addr+yyy zzz> /proc/clkdbg ; cat /proc/clkdbg`
- For example, to adjust driving strength of MIPI, supporter will say: make mipi_tx's 0x10 reg bit6-9 to 0111, the cmd is: `echo reg_read 0x11f60010 > /proc/clkdbg; cat /proc/clkdbg, //mipi_tx base addr is 0x11f60000, read 0x11f60010 first (if read return 0)`
- Let return data 0 bit6-9 or 0111 get 0x1c0: `echo reg_write 0x11f60010 0x1c0> /proc/clkdbg ; cat /proc/clkdbg`

9.4 Test Pattern CMD

- ADB cmd:
 - `echo reg_write "DSI/DSC_base_reg+offset_addr" value> /proc/clkdbg ; cat /proc/clkdbg`
 - Get DSI/DSC base reg with this sop in page 130
 - Offset :
 - Dsi:0x178 or 0x17c
 - Dsc:0x78 or 0x6c · ask it for MTK
 - Value :
 - Dsi: 0xc41
 - Dsc: 0x4000FFFF (blue), or ask for MTK
- Add the cmd in the code:
 - DSI: call `DSI_OUTREG32(NULL, DSI_REG_BASE[0]+offset, value);` at the end of func:Ddp_dsi_trigger
 - DSC: call `DISP_REG_SET(handle, base+offset, value);` at the end of func:dsc_config
 - Offset/value refers to ADB command

9.5 Analyze Reg

- **This is just an example.** Get the correct offset from Coda.

- DSI part (Other Regs are debug or status reg, which can be ignored)

Bit0=1 means DSI start, if 0 means DSI not enable or dump fail

DSI status, ignore it

VSA VBP VFP VACT

HAS_WC HBP_WC
VB_PS_WC/HFP_WC BLLP

Read status reg

DSI Timing control

DSI PHY Timing control

[LK_DDP/DSI]== DSI0 REGS =	0x00000001	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0000:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0001:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0002:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0003:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0004:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0005:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0006:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0007:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0008:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0009:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000a:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000b:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000c:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000d:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000e:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000f:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0010:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0011:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0012:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0013:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0014:	0x00000000	0x00000000	0x00000000	0x00000000

NOTE: HSA WC=HAS*3-10. the same as HBP/HFP

- MIPI TX part (Other registers are for debugging or status, which can be ignored)

Read status reg

Phy vol sel

PLL control 0

PLL control 1/2/3/4

Lane swap setting

[LK_DDP/DSI]- DSI0 CMD REGS =	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0000:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0001:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0002:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0003:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0004:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0005:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0006:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0007:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0008:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x0009:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000a:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000b:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000c:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000d:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000e:	0x00000000	0x00000000	0x00000000	0x00000000
[LK_DDP/DSI]0x000f:	0x00000000	0x00000000	0x00000000	0x00000000

- DSC part (Other regs are debug or status registers, which can be ignored)

DSC control		DSC status		DSC PIC W	
DISP0	DSC REGS	addr	0x14015000		
014015000:	0x00010081	0x00000000	0x00000001	0x00000000	
014015010:	0x00000000	0x00000000	0x01d20578	0x07cf07cf	
014015020:	0x00e902bc	0x40630013	0x000002bc	0x000036b0	
014015030:	0x00010001	0x00000022	0x00000001	0x057802bc	
014015040:	0x00000000	0x00000000	0x00000000	0x00000000	
014015050:	0x00000000	0x00000000	0x00000000	0x00000000	
014015060:	0x00000000	0x00000000	0x00000000	0x00000000	
014015070:	0x00000240	0x05b205b3	0x00000000	0x00000000	
014015080:	0x000c8089	0x027d0200	0x020e0020	0x000d0009	
014015090:	0x03dc057a	0x10d01800	0x2000c03	0x33b0b06	
0140150a0:	0x382a1c0e	0x69625446	0x7b797770	0x00007e7d	
0140150b0:	0x00800880	0xf8c100a1	0xe8e3f0e3	0xe103e0e3	
0140150c0:	0xd943e123	0xd165d945	0xd189d165	0x0000d1ac	
0140150d0:	0x00000000	0x00000000	0x00000000	0x00000000	
0140150e0:	0x00000000	0x00000000	0x00000000	0x00000000	
0140150f0:	0x00000000	0x00000000	0x00000000	0x00000000	
014015100:	0x000002fc	0x0000041c	0x1000244a	0x10002648	
014015110:	0x0c000000	0x00000000	0x00000000	0x000001e8	
014015120:	0x00000000	0x00000000	0x00000000	0x00000000	
014015130:	0x02a6429b	0x02a6429b	0x3fff0000	0x00006d60	
014015140:	0x00e900e9	0x000007d0	0x00000001	0x00000000	
014015150:	0x000e4070	0x00000000	0x00000000	0x00000000	
014015160:	0xa02c2001	0xa02c2001	0x00000000	0x00000000	
014015170:	0x00000000	0x11111111	0x00000000	0x00000000	

DSC PIC H

DSC PPS0-19

10 How to Use MTK max96789 Panel Driver

A driver for the serializer-deserializer max96789+max96752 is provided by MTK, and this driver can be referred to by developers for development.

How to use it:

1. Add birdge-serdes-max96789.ko/panel-serdes-max96789.ko to the KO table (if it does not exist)
2. Set MTK_LCM_LIST_SUPPORT="max96789_dsi_vdo" in LK's project.mk
3. Add birdge-serdes-max96789.ko/panel-serdes-max96789.ko to kleaf (if it does not exist)
4. Add CONFIG_DRM_PANEL_SERDES=m to defconfig (if it does not exist)
5. If enable the hotplug function, need to set CONFIG_ENABLE_SERDES_HOTPLUG=y in defconfig. If use the interrupt method, need set "#define ENABLE_HOTPLUG_INT 1" in birdge-serdes-max96789.c
6. Add the following settings to DTS.
7. Other settings are hard-coded in the driver. If they do not meet the needs, they can be stripped out and written to DTS.

```
&i2c1
{
    status = "okay";
    max96789: max96789@40 {
        compatible = "maxim,max96789";
        reg = <0x40>;
        reset-gpios = <&pio 24 0>;
        interrupt-parent = <&pio>;
        interrupts = <187 IRQ_TYPE_EDGE_RISING>;
        interrupt mode for hotplug
        pinctrl-names = "default";
        init-in-lk = <0>;
        initialize in LK
        setting = <&setting_compatible>;
    };
    &max96789 {
        setting_compatible: compatible-node {
            comp-cmd = <
                0x01 0x1a 0x0b 0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00
                0x04 // format: <r/w><iic addr><cmd len><cmd data>, only for demo
                // r/w: read cmd=0, write cmd=1
                // iic addr: iic addr for read device
                // cmd len: cmd data len for write
                // cmd data: cmd data
                0x00 0x1a 0x0b // read cmd, format: <r/w><iic addr><cmd len>, must
                in the end of "comp-cmd" and must only one read cmd
            >;
            comp-exp {
                comp-setting = <0xf0 0x10 &setting_bt>,
                setting, format: <mask><exception data><panel setting node>
                // mask: the return data of read cmd AND the mask
            };
        };
    };
}
```

```

// exception: the data of exception, if it equal to return data AND mask, will
used this panel setting
// panel setting node: which panel setting with this setting
    <0xff 0xff &setting_hy>; // NOTE: default setting If none of
the previous items match
};
};
setting_bt: setting@1 { // panel setting, must set at least one
    ser-super-frame = <0>; // super frame set to 1, otherwise set to 0
    ser-init-cmd = < // init cmd of serializer, format:
<reg><data><delay>, must set
    0x0001 0x08 0x00
    0x0203 0x00 0x00 // reg: reg addr
    0x1404 0x29 0x00 // data: data to write
    0x1504 0x29 0x00 // delay: how long to delay after send this
cmd, unit: ms
    .....
>;
    ser-deinit-cmd = < // serializer deinit cmd, must set
    0x0010 0x80 0x20
>;
    des-link-status-cmd = < // detect deserializer link status
command, if enable hotplug must set
    0x001f 0x18 0x0
>;
    desdef { // deserializer setting, must set
        des-i2c-addr = <0x4c>; // iic addr of deserializer
        bl-i2c-addr = <0x1a>; // iic addr of backlight
        bl-dummy-i2c-addr = <0x1f>; // if iic addr of backlight same as
touch panel, use it to avoid alloc i2c_client fail
        des-init-command = < // init command for deserializer
        0x01ce 0x5e 2
        0x06ff 0x22 0x0 // must set if hotplug enable, we read
this to find if deserializer was initialized, must same as link-indicate-command
        >;
        link-indicate-command = < // must set if hotplug enable, must same
as one of "des-init-command"
        0x06ff 0x22 0x0
        >;
        bl-on-command = < // command of turn on backlight, get it
from panel vendor
        0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0xd4
        >;
        bl-off-command = < // command of turn off backlight
        0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00 0x85
        >;
        panel-mode-setting { // panel timing, must set
            panel-mode-width = <1920>; // width
            panel-mode-height = <1080>; // height
            panel-mode-hfp = <40>; // HFP
            panel-mode-hsa = <40>; // HSA
            panel-mode-hbp = <80>; // HBP
            panel-mode-vfp = <24>; // VFP
            panel-mode-vsa = <2>; // VSA

```

```

        panel-mode-vbp = <10>; // VBP
        panel-mode-vrefresh = <60>; // frame per second
        panel-mode-pll = <478>; // mipi pll clock, driver will
calculate a PLL according with the panel timing. If any deviation, it will
replace the driver-calculated result.
        panel-mode-lppf = <1>; // set it to 1 when need MIPI enter
LP status per-frame
        panel-mode-width-mm = <129>; // panel physical width, unit: mm
        panel-mode-height-mm = <64>; // panel physical height, unit: mm
    };
};
setting_hy: setting@2 {
    ser-dual-link = <0>;
    ser-init-command = <
        0x0001 0x08 0x00
        .....
    >;
    desdef {
        des-i2c-addr = <0x4c>;
        bl-i2c-addr = <0x1a>;
        des-init-command = <
            0x01ce 0x4e 2
        >;
        bl-on-command = <
            0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0xd4
        >;
        bl-off-command = <
            0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00 0x85
        >;
        panel-mode-setting {
            panel-mode-width = <1920>;
            panel-mode-height = <1080>;
            panel-mode-hfp = <40>;
            panel-mode-hsa = <40>;
            panel-mode-hbp = <80>;
            panel-mode-vfp = <24>;
            panel-mode-vsa = <2>;
            panel-mode-vbp = <10>;
            panel-mode-vrefresh = <60>;
            panel-mode-pll = <478>;
            panel-mode-lppf = <1>;
            panel-mode-width-mm = <129>;
            panel-mode-height-mm = <64>;
        };
    };
};
superframe_setting: setting@3 { // demo of super frame setting
    ser-super-frame = <1>; // must set to 1 when super frame enable
    ser-init-command = <
        0x0002 0x73 0x0
        .....
    >;
};

```

```

ser-timing-command = < // super frame timing setting, must set when
super frame enable
    0x385 0x50 0x0
    0x386 0x02 0x0
    0x387 0x00 0x0
    0x3A5 0x18 0x0 // Note: the order of reg addr cannot
change!
    0x3A7 0x00 0x0
    0x3A6 0xa0 0x0
    0x3A8 0x38 0x0
    0x3A9 0x04 0x0
    0x3AA 0x50 0x0
    0x3AC 0x0A 0x0
    0x3AB 0x00 0x0
    0x3AD 0x00 0x0
    0x3AE 0x0f 0x0
>;
serdes-dual-setting-command = < // remap deserializer addr, get more
info from MAXIAM
    0 0x0010 0x21 255
    1 0x0000 0x90 0
    2 0x0073 0x31 0
    2 0x0042 0x36 0
    2 0x0043 0x34 0
    0 0x0010 0x22 255
    1 0x0000 0x94 0
    3 0x0073 0x32 0
    3 0x0042 0x38 0
    3 0x0043 0x34 0
    0 0x0010 0x23 255
    3 0x0050 0x01 0
>;
ser-lut-command = < // lut command for asymmetric super
frame
>;
ser-deinit-command = <
    0x0010 0x80 0x20
>;
des-link-status-command = <
    0x001f 0x18 0x0
>;
desdef { // deserializer default setting, must set
    des-i2c-addr = <0x4c>; // deserializer default i2c addr, must set
    bl-i2c-addr = <0x1a>; // deserializer backlight default i2c addr
};
desa { // linka deserializer config, must set when super
frame enable
    des-i2c-addr = <0x48>; // linka des i2c addr, it was remap by
serializer
    bl-i2c-addr = <0x1b>; // linka backlight i2c addr, it was remap by
serializer
    des-init-command = < // linka deserializer initialize command
        0x01ce 0x5e 2
        0x06ff 0x22 0x0 // must set if hotplug enable, we read this

```

```

to find if deserializer was initialized, must same as link-indicate-command
>;
link-indicate-command = < // must set if hotplug enable, must same
as one of "des-init-command"
0x06ff 0x22 0x0
>;
bl-on-command = < // the meaning same as single link
0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0xd4
>;
bl-off-command = < // the meaning same as single link
0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00 0x85
>;
panel-mode-setting { // the meaning same as single link
setting
    panel-mode-width = <1920>;
    panel-mode-height = <1080>;
    panel-mode-hfp = <40>; // Note: it must be an integer
multiple of 4
    panel-mode-hsa = <41>; // Note: The sum of it and the value
of desb minus 10 must be an integer multiple of 4
    panel-mode-hbp = <79>; // Note: The sum of it and the value
of desb minus 10 must be an integer multiple of 4
    panel-mode-vfp = <24>; // Note: must be desa_vfp/desb_vfp =
desa_height/desb_height in asymmetrice mode
    panel-mode-vsa = <3> // Note: must be desa_vsa/desb_vsa =
desa_height/desb_height in asymmetrice mode;
    panel-mode-vbp = <9>; // Note: must be desa_vbp/desb_vbp =
desa_height/desb_height in asymmetrice mode
    panel-mode-vrefresh = <60>;
    panel-mode-pll = <478>; // super frame not used it
    panel-mode-lppf = <1>; // super frame must set to 1
    panel-mode-width-mm = <129>;
    panel-mode-height-mm = <64>;
};
};
desb { // the meaning same as linka, must set when super
frame enable
    des-i2c-addr = <0x4a>;
    bl-i2c-addr = <0x1c>;
    des-init-command = <
0x01ce 0x5e 2
0x06ff 0x22 0x0
>;
link-indicate-command = <
0x06ff 0x22 0x0
>;
bl-on-command = <
0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0xd4
>;
bl-off-command = <
0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00 0x85
>;
panel-mode-setting {
    panel-mode-width = <1920>;

```

```

        panel-mode-height = <1080>;
        panel-mode-hfp = <40>;
        panel-mode-hsa = <41>;
        panel-mode-hbp = <79>;
        panel-mode-vfp = <24>;
        panel-mode-vsa = <3>;
        panel-mode-vbp = <9>;
        panel-mode-vrefresh = <60>;
        panel-mode-pll = <478>;
        panel-mode-lppf = <1>;
        panel-mode-width-mm = <129>;
        panel-mode-height-mm = <64>;
    };
};

};

&dsi0 {
    status = "okay";
    #address-cells = <1>;
    #size-cells = <0>;
    panel1@0 {
        compatible = "lcm,dsi,max96789"; // panel setting
        reg = <0>;
        ser = <&max96789>; // panel link with which serializer,
must set
        port {
            panel_in1: endpoint {
                remote-endpoint = <&dsi_out>;
            };
        };
    };
};

```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.